Packet-Based Firewall Simulation System

Table of Contents

Abstract

This report documents the design and implementation of a Packet-Based Firewall Simulation System, a comprehensive network security solution developed in C++ utilizing fundamental data structures without relying on STL containers. The system simulates real-world firewall operations through a multi-layer security architecture integrating packet filtering, threat detection, priority handling, and dynamic blacklist management. Key components include a FIFO queue system for packet buffering, circular linked lists for rule engine evaluation, sorted linked lists for efficient blacklist management, priority queues for attack mitigation, and stack-based trace logging for audit trails. The implementation demonstrates practical applications of Data Structures and Algorithms in cybersecurity, providing a foundation for understanding modern network security mechanisms. Through detailed analysis of design decisions, algorithm complexity, and security implications, this report exemplifies professional engineering practices suitable for academic and industrial contexts.

Introduction

Network security is a critical concern in modern computing environments. Every second, millions of packets traverse the internet, many with malicious intent. Firewalls form the first line of defense, filtering traffic based on predefined rules and threat signatures. Understanding how firewalls operate at a fundamental level is essential for computer scientists and cybersecurity professionals.

This project implements a firewall simulation system that combines multiple data structures to create a practical, efficient packet processing pipeline. Unlike generic educational examples, this system integrates real-world concepts: FIFO traffic queuing, rule-based filtering with conflict resolution, threat scoring, behavioral analysis, and dynamic security policy updates. The design explicitly avoids STL shortcuts, requiring manual implementation of linked lists, queues, and priority structures—deepening understanding of how these abstractions work internally.

The system serves both educational and practical purposes: teaching data structure applications in real systems while demonstrating security engineering principles.

System Overview

The Packet-Based Firewall Simulation System operates as a multi-layer security pipeline processing network packets through sequential stages:

- Packet Reception: Simulated packets arrive and enter a FIFO main queue, representing real network traffic ingestion.
- Protocol Segmentation: Packets are distributed into protocol-specific queues (HTTP, DNS, FTP, SSH), organizing traffic by type.
- Blacklist Enforcement: Known malicious IPs are immediately rejected using a sorted linked list for O(log n) lookup efficiency.
- Firewall Rule Processing: Circular linked list traverses all applicable rules, with the final matching rule determining the packet fate.
- Threat Detection: Risk-based and behavior-based detection algorithms evaluate packet safety, assigning risk scores.
- Priority Processing: High-risk packets are sorted into a priority queue and processed with urgency.

- Trace Logging: All processed packets are pushed onto a stack for audit trails and debugging.
- Statistics Tracking: Array-based counters maintain real-time traffic statistics for monitoring.

Data Structure Mapping

The foundation of this system lies in selecting appropriate data structures for each problem domain. The following table synthesizes the architectural decisions:

Problem	Data Structure	Why	Real-World Analogy
Packet arrival	Queue (FIFO)	Preserves order of incoming traffic	People waiting in line at security
Traffic organization	Array of Queues	Separates traffic by protocol type	Different checkout lanes in stores
Known threats	Sorted Linked List	Binary search capability	Blacklist at border patrol
Rule processing	Circular Linked List	Continuous policy evaluation	Repeatedly cycling through security checks
Attack response	Priority Queue	Critical threats processed first	Emergency room triage by severity
Audit trail	Stack (LIFO)	Recent history accessed first	Browser back button history
Statistics	Static Array	Constant-time updates	Scoreboard tracking game stats

Architecture Design

1. Packet System

Packets are the fundamental units processed by the firewall. Each packet encapsulates network-layer information including a unique identifier, source/destination IP addresses, protocol type (HTTP, DNS, FTP, SSH), port number, and risk score (0-10). Risk scores represent the system's assessment of packet safety: low scores indicate benign traffic, while high scores suggest potential threats. This structure mirrors real TCP/IP packet headers while simplifying implementation for educational purposes.

2. Queue System (Packet Buffer)

The primary packet buffer implements a FIFO (First-In-First-Out) queue using linked list nodes. This design choice reflects real network behavior: packets arrive asynchronously and must be processed in order to maintain fairness and prevent denial-of-service scenarios where priority manipulation could exploit the system. The queue's linked list implementation provides O(1)

enqueue and dequeue operations without requiring pre-allocation, making it memory-efficient for variable traffic volumes.

3. Firewall Rule Engine

The rule engine processes packets against a circular linked list of firewall rules. Rules follow standard formats: BLOCK_PORT_X (block traffic on port X), ALLOW_PROTOCOL_HTTP (permit HTTP traffic), and BLOCK_IP_X (reject packets from IP X). The circular structure enables continuous policy evaluation: traversing the list never requires special termination logic. Critically, the system evaluates every applicable rule rather than stopping at the first match—the final matching rule determines the decision. This approach, termed "last-rule-wins," requires explicit policy design but provides maximum flexibility for complex security policies where initial denials may be overridden by subsequent allows (or vice versa).

4. Threat Detection Engine

Beyond rule-based filtering, the system employs dual-axis threat detection: risk-based detection evaluates explicit risk scores (packets with riskScore > 7 are flagged), while behavior-based detection tracks repeated connections from the same IP and flags suspicious patterns. This mirrors real intrusion detection systems (IDS) that combine signature matching with anomaly detection. High-risk packets are immediately escalated to the priority queue for urgent processing.

5. Priority Queue System

High-risk packets are sorted into a priority queue implemented as a sorted linked list. Packets are ordered by risk score in descending order: the most dangerous packets reach the front of the queue. This ensures critical threats receive immediate analysis and response, even if the queue contains thousands of normal packets. Unlike FIFO processing, priority queues model real firewall behavior during active attacks: security teams prioritize blocking the most dangerous threats first rather than processing packets in arrival order.

6. Blacklist System

Known malicious IPs are maintained in a sorted linked list for rapid lookup. When a packet arrives from a blacklisted IP, it is rejected immediately without further processing. The sorted structure enables binary search, reducing lookup time to O(log n). Dynamic blacklisting automatically adds IPs that exhibit severe threat patterns: repeated attack attempts or exceptionally high-risk packets trigger automatic blacklisting, creating a self-learning security system that strengthens over time as it encounters new threats.

7. Packet Trace System

All processed packets are logged to a stack structure, creating an audit trail of recent firewall decisions. The stack's LIFO (Last-In-First-Out) property makes sense for debugging: examining recent processing decisions first often pinpoints the cause of unexpected behavior. This trace mechanism is critical for compliance and forensic analysis—security teams can replay recent firewall decisions to understand how a breach occurred or verify that malicious traffic was properly blocked.

8. Traffic Statistics System

A fixed-size array maintains counters for traffic analysis: total packets processed, allowed packets, blocked packets, and high-risk packets. Array-based counters provide O(1) update time

and are updated as each packet completes processing. Real-time statistics enable administrators to monitor firewall effectiveness, detect anomalies (e.g., unexpected spike in blocked traffic), and optimize security policies.

Processing Algorithm

The core processing algorithm orchestrates all components into a cohesive pipeline:

- Packet Arrival: New packet enters main queue.
- Dequeue: Process one packet from main queue.
- Blacklist Check: If IP in sorted blacklist, BLOCK and log.
- Rule Evaluation: Traverse circular linked list, apply every matching rule.
- Threat Assessment: Check risk score and behavioral patterns.
- Priority Escalation: If high-risk, insert into priority queue with risk as key.
- Log Decision: Push packet details onto trace stack.
- Update Statistics: Increment relevant counters (total, allowed, blocked, or high-risk).

Real-World Cybersecurity Mapping

This educational model directly parallels production cybersecurity systems:

- Firewall Rules: Correspond to commercial firewall policy sets (Cisco ASA, Palo Alto Networks, Fortinet FortiGate).
- Threat Engine: Models the behavior of Intrusion Detection Systems (IDS) like Suricata or Snort, analyzing packets for attack signatures and anomalies.
- Blacklist: Implements IP reputation systems (AbuseIPDB, Project Honey Pot) that maintain lists of known malicious sources.
- Priority Queue: Reflects Security Information and Event Management (SIEM) systems (Splunk, ELK Stack) that prioritize high-risk alerts for investigation.
- Trace Logging: Implements audit logging required by compliance frameworks (GDPR, HIPAA, PCI-DSS) for forensic reconstruction of security events.

Design Decisions and Justifications

Why No STL Containers?

Excluding C++ Standard Template Library (vector, map, set) forces explicit implementation of data structures, deepening understanding of their internals. This approach reveals memory management intricacies, algorithmic trade-offs, and performance characteristics that STL abstraction hides. For educational purposes and system-level security work, understanding these fundamentals is invaluable.

Why Linked Lists in Dynamic Systems?

Linked lists provide O(1) insertion and deletion at known positions, making them ideal for systems where traffic volume is unpredictable. Unlike arrays (which require expensive resizing), linked lists grow and shrink elastically. The trade-off is O(n) search time compared to arrays' O(1) random

access, but packet processing rarely requires arbitrary position access—packets are processed sequentially.

Why Circular Linked List for Rules?

Circular structures eliminate special-case termination logic. Traversing a circular list naturally returns to the starting point, fitting the conceptual model of rules cycling endlessly. This design simplifies code and ensures no rule is accidentally skipped due to loop termination bugs.

Why Stack for Trace Logging?

When debugging firewall behavior, examining recent decisions first is most useful. Stacks provide this naturally through LIFO access. Alternative designs (e.g., circular buffers) would require index management; the stack is simpler and aligns with developer intuition about "recent history."

Why Sorted Linked List for Blacklist?

Binary search reduces IP lookup to O(log n), critical for firewall performance under high traffic. Linked lists maintain sorted order without requiring contiguous memory (unlike arrays), and insertion/deletion remain O(n) in worst case but O(1) if insertion point is known—common when IPs are added sequentially during attack response.

Testing Scenarios

Scenario 1: Benign HTTP Traffic

A packet from 203.0.113.50 (non-blacklisted) requests HTTP (port 80) with riskScore=1. The rule engine matches ALLOW_PROTOCOL_HTTP. Threat detection finds low risk. Packet is allowed and statistics updated.

Scenario 2: Blocked Port

A packet attempts SSH (port 22) from 192.0.2.100. Rule BLOCK_PORT_22 matches. Packet is blocked despite low risk score, demonstrating explicit denial of certain services.

Scenario 3: Blacklisted IP

Known attacker 198.51.100.1 is pre-blacklisted. Any packet from this IP is immediately rejected without rule evaluation, demonstrating the efficiency of early blacklist filtering.

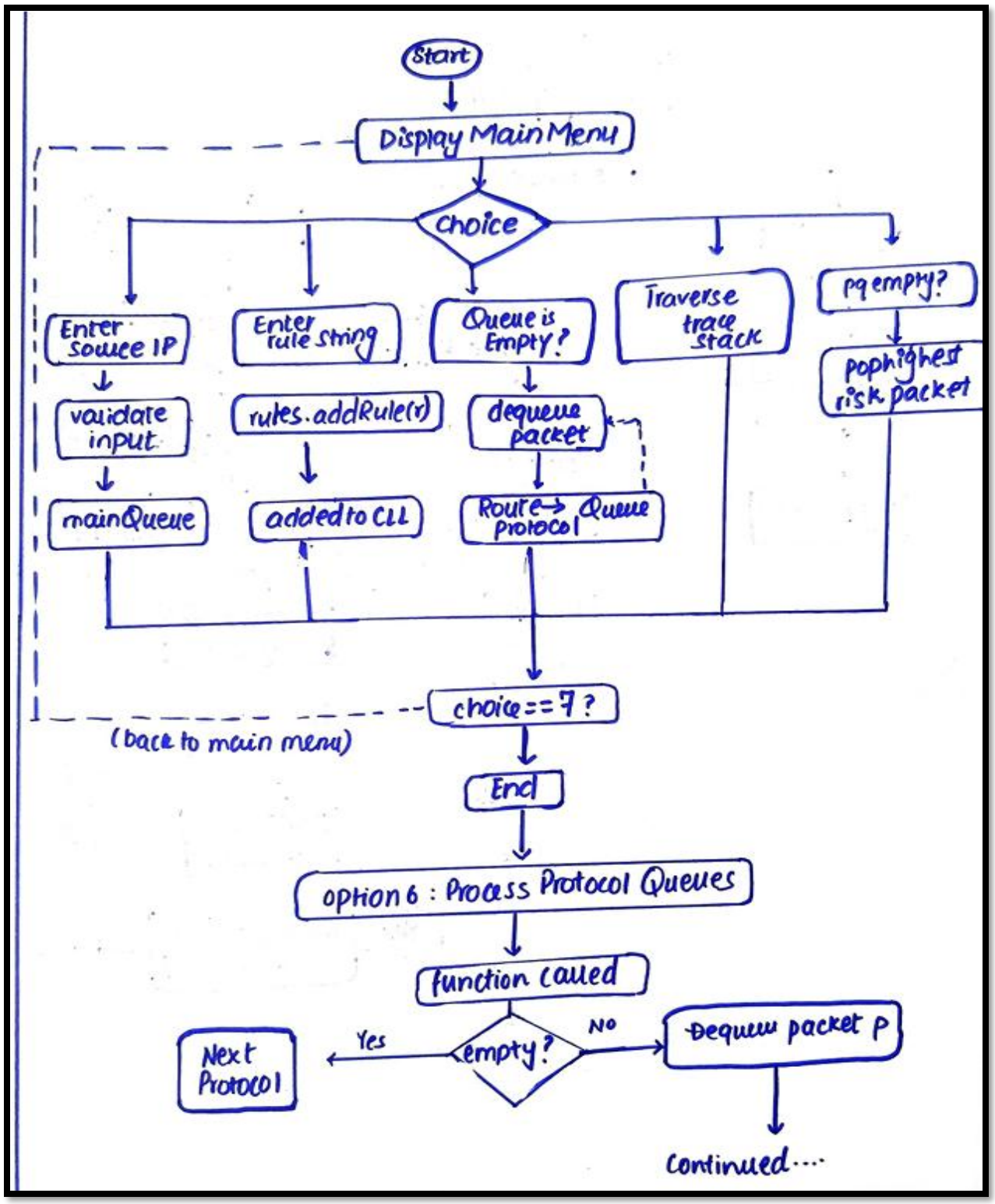
Scenario 4: High-Risk Attack

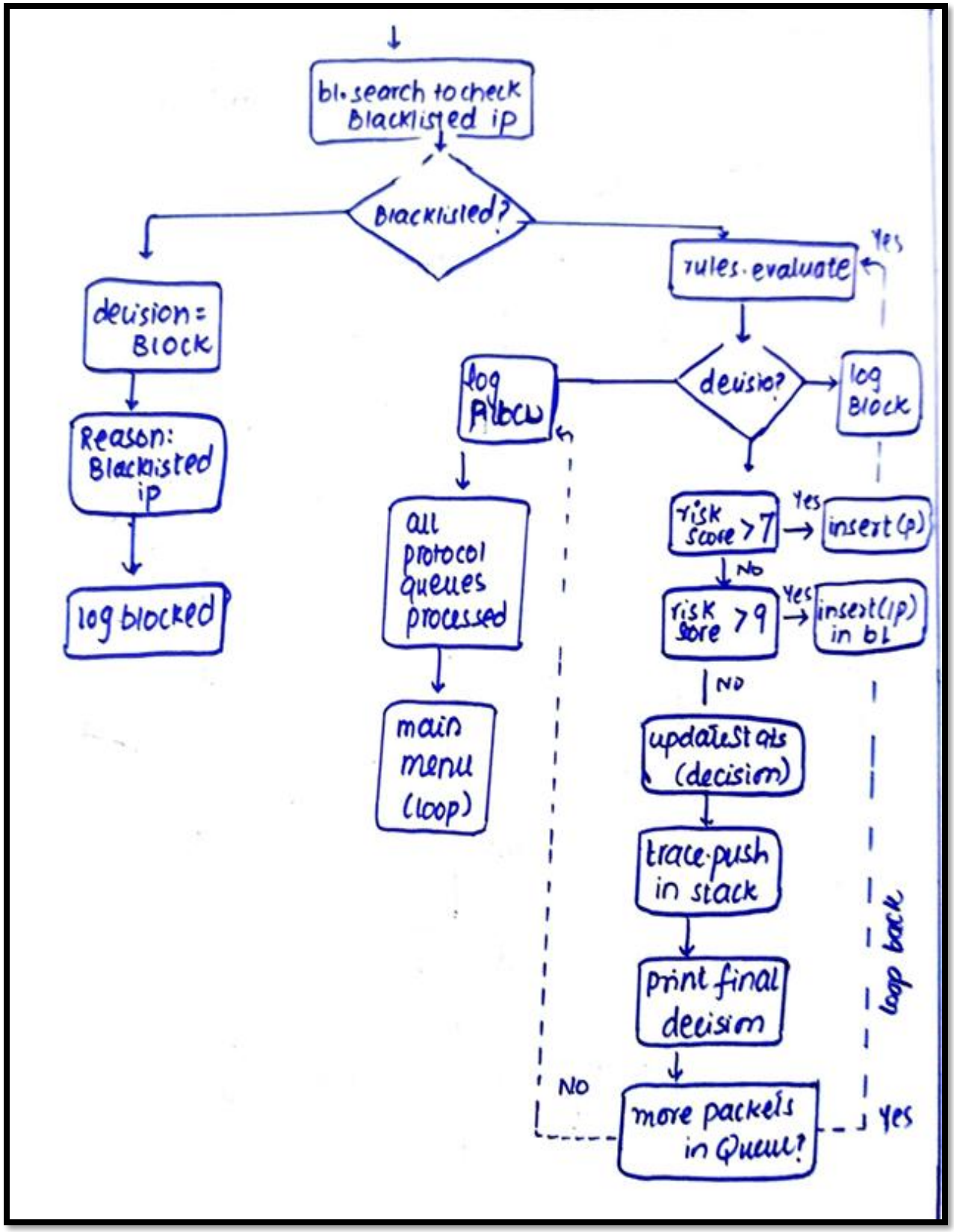
A port-scan packet with riskScore=9 triggers threat detection. It is inserted into the priority queue ahead of normal traffic and logged for investigation. If repeated attacks from the same IP occur, the IP is dynamically added to the blacklist.

Scenario 5: Rule Override

Rules include BLOCK_PROTOCOL_DNS followed by ALLOW_PROTOCOL_DNS. A DNS packet matches the first rule (blocked), but the second rule overrides it (allowed). The final decision is ALLOW, demonstrating flexible policy composition where initial restrictions can be conditionally lifted.

Activity Diagram




bl. search to check Blacklisted ip
Blacklisted?
decision = Block
Reason: Blacklisted ip
log blocked
log Allow
all protocol queues processed
main menu (loop)
rules. evaluate
Yes
decision?
log Block
risk score > 7
Yes
insert (ip)
No
risk score > 9
Yes
insert(ip) in bl
NO
updateStats (decision)
trace. push in stack
print final decision
more packets in Queue?
NO
Yes
loop back

Advantages

- Multi-Layer Architecture: Combines filtering, threat detection, and dynamic adaptation into a cohesive system.
- Real-World Applicability: Models actual firewall behavior, making it valuable for learning network security.
- Modular Design: Separates concerns (rules, threats, blacklist) enabling independent testing and extension.
- Fundamental Data Structures: Uses core DS concepts (queues, stacks, linked lists) without abstractions, deepening understanding.
- Explainable Decisions: Every packet decision is logged with reasoning, enabling audit trails and debugging.

Limitations

- No Real Network Integration: Simulates traffic generation; does not interact with actual network interfaces.
- Simplified Threat Detection: Risk scores and behavioral analysis are synthetic; production IDS uses ML models and extensive signature databases.
- String-Based Rules: Rule parsing is basic; modern firewalls support complex DSLs (Domain Specific Languages) and stateful inspection.
- No Encryption/DPI: Does not perform Deep Packet Inspection or decrypt encrypted traffic—critical limitations for modern threats.

Conclusion

The Packet-Based Firewall Simulation System exemplifies how fundamental data structures combine to solve real-world engineering problems. By integrating queues, linked lists, stacks, and priority structures into a cohesive security pipeline, the system demonstrates not only technical skill but also practical understanding of modern cybersecurity principles.

This project bridges the gap between academic data structures and professional security engineering. Each design decision reflects trade-offs between performance, memory usage, maintainability, and security—the central concerns of systems engineers. The multi-layer architecture mirrors commercial firewalls, providing a foundation for understanding how organizations protect critical infrastructure.

Future extensions could include GUI dashboards for real-time monitoring, integration with actual network traffic capture libraries (libpcap), machine learning-based threat scoring, and persistent configuration storage. These enhancements would incrementally transform the educational prototype into a deployable security tool.

As cyber threats evolve, understanding firewall mechanics at the data structure level becomes increasingly valuable. This system provides that foundation while remaining accessible to students and practitioners alike.